1. Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug?

Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.

Water Jug Problem

We are given two jugs of capacities 4 gallons and 3 gallons, without measurement markings. The objective is to obtain exactly 2 gallons of water in the 4-gallon jug using a pump, transferring water between the jugs, and emptying water onto the ground. No additional measuring devices are used.

Method

The problem is solved using the State Space Search technique. Each state is represented as \((x,y)\), where \(x\) represents the water in the 4-gallon jug and \(y\) represents the water in the 3-gallon jug.

Initial state: \((0,0)\)

Goal state: \((2,0)\)

The operations allowed are filling a jug, emptying a jug, and transferring water from one jug to another.

Procedure

Step	Action	State (4-gallon, 3-gallon)
1	Fill the 3-gallon jug.	(0, 3)
2	Pour water into the 4-gallon jug.	(3, 0)
3	Fill the 3-gallon jug again.	(3, 3)
4	Pour water into the 4-gallon jug until it is full.	(4, 2)
5	Empty the 4-gallon jug.	(0, 2)
6	Transfer the remaining 2 gallons into the 4-gallon jug.	(2, 0)

Explanation

First, the 3-gallon jug is filled and poured into the 4-gallon jug. It is filled again, and the water is transferred until the 4-gallon jug becomes full. This leaves exactly 2 gallons in the 3-gallon jug. The 4-gallon jug is emptied, and the remaining water is transferred into it.

Result

Thus, the Water Jug problem is solved successfully using state space search, obtaining exactly 2 gallons of water in the 4-gallon jug without any additional measuring device.

2. Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.

i. What are the percept's for this agent?

ii. Characterize the operating environment.

iii. What are the actions the agent can take?

iv. How can one evaluate the performance of the agent?

v. What sort of agent architecture do you think is most suitable for this agent?

Why?

Mars Rover: Intelligent Agent

A Mars Rover is an intelligent robotic agent designed to explore the surface of Mars, collect soil and rock samples, analyze them, and transmit scientific information to Earth.

i. What are the percepts for this agent?

The percepts are the information received by the Mars Rover through its sensors and instruments.

- Cameras: Capture images of the Martian surface and surroundings.
- Proximity sensors: Detect nearby rocks, obstacles, and hazards.
- Navigation sensors: Determine the rover's position, direction, and movement.
- Scientific instruments: Analyze soil, rocks, minerals, and chemical compositions.
- Environmental sensors: Measure temperature, atmospheric pressure, and other environmental conditions.
- Power sensors: Monitor battery level and energy availability.

These percepts help the rover understand its surroundings and make suitable decisions.

ii. Characterize the operating environment.

The Mars Rover operates in an environment with the following characteristics:

- Partially observable: The rover cannot observe the entire Martian surface at once.
- Partially deterministic: Actions may have uncertain outcomes because of loose soil, rocks, and unpredictable terrain.
- Sequential: Each action affects future actions and decisions.
- Dynamic: Environmental conditions and the rover's state can change during operation.

- Continuous: Movement, distance, temperature, and time involve continuous values.
- Single-agent: The rover generally makes its own exploration decisions, although it may receive instructions from Earth.

Therefore, the Mars environment is challenging and requires intelligent decision-making.

iii. What are the actions the agent can take?

The Mars Rover can perform the following actions:

- Move forward, backward, and turn left or right.
- Stop or avoid obstacles.
- Capture images of rocks and the surrounding terrain.
- Collect soil and rock samples using its robotic arm.
- Analyze samples using scientific instruments.
- Select suitable locations for further exploration.
- Monitor its battery and manage power consumption.
- Store scientific data and transmit information to Earth.
- Return to a safe position when necessary.

These actions help the rover explore Mars safely and collect useful scientific information.

iv. How can one evaluate the performance of the agent?

The performance of the Mars Rover can be evaluated using the following criteria:

- Exploration efficiency: Amount of useful terrain explored.
- Sample collection: Number and quality of soil and rock samples collected.
- Scientific accuracy: Accuracy of sample analysis and scientific measurements.
- Safety: Ability to avoid obstacles and prevent damage.
- Energy efficiency: Effective use of battery power.
- Communication: Successful transmission of images and scientific data to Earth.
- Mission completion: Achievement of the assigned exploration objectives within the available time and resources.

A rover performs well when it collects valuable scientific information, completes its objectives, and operates safely while conserving energy.

v. What sort of agent architecture is most suitable for this agent? Why?

The model-based, goal-based, and utility-based agent architecture is most suitable for a Mars Rover. A hybrid architecture combining these approaches is ideal.

- Model-based agent: Maintains an internal representation of the terrain, obstacles, rover position, and environmental conditions.
- Goal-based agent: Selects actions to achieve goals such as reaching a target location or collecting a sample.
- Utility-based agent: Chooses the best action by balancing scientific value, safety, distance, time, and energy consumption.
- Learning capability: Helps improve navigation and sample-selection decisions from previous observations and experiences.

Reason: The Mars Rover operates in an uncertain and partially observable environment. It must plan its movements, avoid hazards, analyze samples, and conserve power. A hybrid architecture allows it to use its internal model, achieve mission goals, and select actions that maximize scientific value while maintaining safety.

Result

Thus, the Mars Rover is an intelligent agent that perceives its environment through sensors, analyzes samples, selects suitable actions, and evaluates its performance based on scientific success, safety, and resource efficiency. A hybrid model-based, goal-based, and utility-based architecture is best suited for autonomous Mars exploration.

3. Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.

3. Eight Queens Problem

The 8-Queens problem is a classic Artificial Intelligence problem in which eight queens must be placed on an \(8 \times 8\) chessboard such that no queen attacks another queen. In chess, a queen can attack horizontally, vertically, and diagonally. Therefore, no two queens should share the same row, column, or diagonal.

Problem Formulation

1. Initial State

The chessboard is initially empty, with no queens placed on it.

2. State Space

The state space consists of all possible configurations of queens on the chessboard, from an empty board to a board containing eight queens.

3. Actions

Place a queen in an empty square such that it does not attack any previously placed queen. A common approach is to place one queen in each column.

4. Transition Model

The transition model describes the result of placing a queen in a valid square. Each action adds one queen to the board while maintaining the constraints for the queens already placed.

5. Goal Test

The goal test checks whether all eight queens have been placed and no two queens attack each other. If all conditions are satisfied, the solution is reached.

6. Path Cost

Each queen placement can be assigned a cost of 1. The total cost of placing eight queens is 8. The main objective is to find a valid arrangement rather than minimize the number of moves.

Search Strategies

The 8-Queens problem can be solved using the following search strategies:

- Breadth-First Search (BFS): Explores configurations level by level. It may require considerable memory because the search space is large.
- Depth-First Search (DFS): Places queens one at a time and backtracks whenever a conflict occurs.
- Backtracking Search: Checks each placement and rejects any configuration that violates the constraints. This is an efficient approach for the 8-Queens problem.
- Heuristic Search: Uses information about conflicts to guide the search toward a valid solution.

Example of a Valid Solution

One valid arrangement of eight queens is represented by the following row positions for columns 1 through 8:

\[[1,5,8,6,3,7,2,4] \]

This means that the queen in column 1 is placed in row 1, the queen in column 2 is placed in row 5, and so on.

8-Queens Solution

Q represents a queen.

In this arrangement, every column and every row contains exactly one queen, and no two queens share a diagonal.

Result

Thus, the 8-Queens problem is formulated as a state-space search problem with an initial state, possible actions, a transition model, a goal test, and a path cost. By using depth-first search

with backtracking, a valid arrangement of eight queens can be found in which no queen attacks any other queen.

4. A customer wants to travel from the one location to another location using OLA Cab

booking mobile application. The customer can select the any of the cab type as mini,

micro, sedan, shared, prime and so on based on his comfort to reach the destination.

Identify what type of problem-solving agent can be used and write the pseudocode

for the above problem.

4. OLA Cab Booking Application

Problem Understanding

A customer wants to travel from a source location to a destination using the OLA cab booking mobile application. The customer can choose different cab types, such as Mini, Micro, Sedan, Shared, and Prime, based on comfort, cost, and availability. The system must identify a suitable cab and help the customer reach the destination.

Type of Problem-Solving Agent

A goal-based problem-solving agent can be used for this problem.

The agent identifies the customer's destination as the goal and searches for a suitable travel option. It considers the source location, destination, cab type, estimated fare, travel time, and availability to select an appropriate option.

Problem Formulation

- Initial state: Customer is at the source location.
- Goal state: Customer reaches the destination.
- Actions: Enter locations, select a cab type, check availability, book a cab, and travel to the destination.
- Transition model: The selected cab moves the customer from the source to the destination.
- Goal test: Check whether the customer has reached the destination.
- Path cost: Fare, travel time, distance, and customer comfort.

Pseudocode

```
BEGIN

  INPUT source_location
  INPUT destination_location
```

```
DISPLAY "Select Cab Type"
DISPLAY "1. Micro"
DISPLAY "2. Mini"
DISPLAY "3. Sedan"
DISPLAY "4. Shared"
DISPLAY "5. Prime"

INPUT cab_type

IF source_location = destination_location THEN
   DISPLAY "Source and destination are the same"
ELSE
   CHECK cab availability

   IF cab is available THEN
      CALCULATE estimated fare
      CALCULATE estimated travel time

      DISPLAY cab_type
      DISPLAY estimated fare
      DISPLAY estimated travel time

      ASK customer to confirm booking

      IF customer confirms THEN
         BOOK cab
         DISPLAY "Cab booked successfully"
         TRAVEL to destination

         IF destination is reached THEN
```

```
            DISPLAY "Trip completed successfully"
          ENDIF
        ELSE
          DISPLAY "Booking cancelled"
        ENDIF

      ELSE
        DISPLAY "Cab not available"
      ENDIF
    ENDIF

END
```

Result

Thus, a goal-based problem-solving agent can be used in the OLA cab booking application to identify a suitable cab, plan the journey, and help the customer reach the destination. It can also consider fare, travel time, availability, and customer preferences when selecting a cab.

5. A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path from the starting warehouse S to the destination warehouse G using the Uniform Cost Search (UCS) algorithm.

5. Uniform Cost Search (UCS) Algorithm

Problem Understanding

A logistics company transports packages between warehouses. Each route between warehouses has a travel cost based on fuel consumption and time. The objective is to find the least-cost path from the starting warehouse S to the destination warehouse G using the Uniform Cost Search (UCS) algorithm.

Method

Uniform Cost Search is an uninformed search algorithm that expands the node with the lowest cumulative path cost first. It uses a priority queue to select the next warehouse to explore.

Problem Formulation

- Initial state: Starting warehouse S.

- Goal state: Destination warehouse G.
- Actions: Travel from one warehouse to an adjacent warehouse.
- Path cost: Total cost of all routes travelled.
- Goal test: Check whether the current warehouse is G.
- Objective: Find the path with the minimum total cost.

Example Input

Consider the following delivery network with route costs:

Route	Cost
S → A	2
S → B	5
A → C	2
A → G	7
B → G	2
C → G	3

UCS Working

Step	Node selected	Cumulative cost	Explanation
1	S	0	Start from S
2	A	2	Lowest-cost route from S
3	C	4	Reach C through A
4	B	5	Next lowest-cost node
5	G	7	Reach G through B

The possible paths include:

- S → A → G = 2 + 7 = 9
- S → A → C → G = 2 + 2 + 3 = 7
- S → B → G = 5 + 2 = 7

Therefore, the minimum cost is 7, with two equally optimal paths.

Pseudocode

BEGIN

```
    Create an empty priority queue
    Insert S with cost 0
    Mark all nodes as unvisited

    WHILE priority queue is not empty

        Remove the node with the lowest cost

        IF node is G
            Display the path and total cost
            STOP
        ENDIF

        Mark node as visited

        FOR each adjacent node
            Calculate new cost

            IF adjacent node is not visited
                Insert adjacent node
                with the new cumulative cost
            ENDIF
        ENDFOR

    ENDWHILE

    Display "Path not found"

END
```

Result

Thus, the Uniform Cost Search algorithm finds a least-cost path from warehouse S to warehouse G with a minimum total cost of 7. The optimal paths are S → A → C → G and S → B → G.